5.4 Separate Compilation--Logistics

When a program consists of many functions, it can be convenient to split them up into several source files. Among other things, this means that when a change is made, only the source file containing the change has to be recompiled, not the whole program.

The job of putting the pieces of a program together and producing the final executable falls to a tool called the *linker*. (We may or may not need to invoke the linker explicitly; a compiler often invokes it automatically, as needed.) The linker looks through all of the pieces making up the program, sorting out the external declarations and defining instances. The compiler has noted the definitions made by each source file, as well as the declarations of things used by each source file but (presumably) defined elsewhere. For each thing (global variable or function) used but not defined by one piece of the program, the linker looks for another piece which does define that thing.

The logistics of writing a program in several source files, and then compiling and linking all of the source files together, depend on the programming environment you're using. We'll cover two possibilities, depending on whether you're using a traditional command-line compiler or a newer integrated development environment (IDE) or other graphical user interface (GUI) compiler.

When using a command-line compiler, there are usually two main steps involved in building an executable program from one or more source files. First, each source file is compiled, resulting in an *object file* containing the machine instructions (generated by the compiler) corresponding to just the code in that source file. Second, the various object files are *linked* together, with each other and with *libraries* containing code for functions which you did not write (such as `printf`), to produce a final, executable program.

Under Unix, the `cc` command can perform one or both steps. So far, we've been using extremely simple invocations of `cc` such as

```
cc -o hello hello.c
```

This invocation compiles a single source file, `hello.c`, links it, and places the executable in a file named `hello`.

Suppose we have a program which we're trying to build from three separate source files, `x.c`, `y.c`, and `z.c`. We could compile all three of them, and link them together, all at once, with the command

```
cc -o myprog x.c y.c z.c
```

Alternatively, we could compile them separately: the `-c` option to `cc` tells it to compile only, but not to link. Instead of building an executable, it merely creates an object file, with a name ending in `.o`, for each source file compiled. So the three commands

```
cc -c x.c
cc -c y.c
cc -c y.c
```

would compile `x.c`, `y.c`, and `z.c` and create object files `x.o`, `y.o`, and `z.o`. Then, the three object files could be linked together using

```
cc -o myprog x.o y.o z.o
```

When the `cc` command is given an `.o` file, it knows that it does not have to compile it (it's an object file, already compiled); it just sends it through to the link process.

Above we mentioned that the second, linking step also involves pulling in library functions. Normally, the functions from the Standard C library are linked in automatically. Occasionally, you must request a library manually; one common situation under Unix is that the math functions tend to be in a separate math library, which is requested by using `-lm` on the command line. Since the libraries must typically be searched *after* your program's own object files are linked (so that the linker knows which library functions your program uses), any `-l` option must appear *after* the names of your files on the command line. For example, to link the object file `mymath.o` (previously compiled with `cc -c mymath.c`) together with the math library, you might use

```
cc -o mymathprog mymath.o -lm
```

(The `l` in the `-l` option is the lower case ell, for **l**ibrary; it is *not* the digit `1`.)

Everything we've said about `cc` also applies to most other Unix C compilers. (Many of you will be using `gcc`, the FSF's GNU C Compiler.)

There are command-line compilers for MS-DOS systems which work similarly. For example, the Microsoft C compiler comes with a `CL` (``compile and link'') command, which works almost the same as Unix `cc`. You can compile and link in one step:

```
cl hello.c
```

or you can compile only:

```
cl /c hello.c
```

creating an object file named `hello.obj` which you can link later.

The preceding has all been about command-line compilers. If you're using some kind of integrated development environment, such as Borland's Turbo C or the Microsoft Programmer's Workbench or Visual C or Think C or Codewarrior, most of the mechanical details are taken care of for you. (There's also less I can say here about these environments, because they're all different.) Typically you define a ``project,'' and there's a way to specify the list of files (modules) which make up your project. The modules might be source files which you typed in or obtained elsewhere, or they might be source files which you created within the environment (perhaps by requesting a ``New source file,'' and typing it in). Typically, the programming environment has a single ``build'' button which does whatever's required to build (and perhaps even execute) your program. There may also be configuration windows in which you can specify compiler options (such as whether you'd like it to accept C or C++). ``See your manual for details.''

Read sequentially: prev next up top